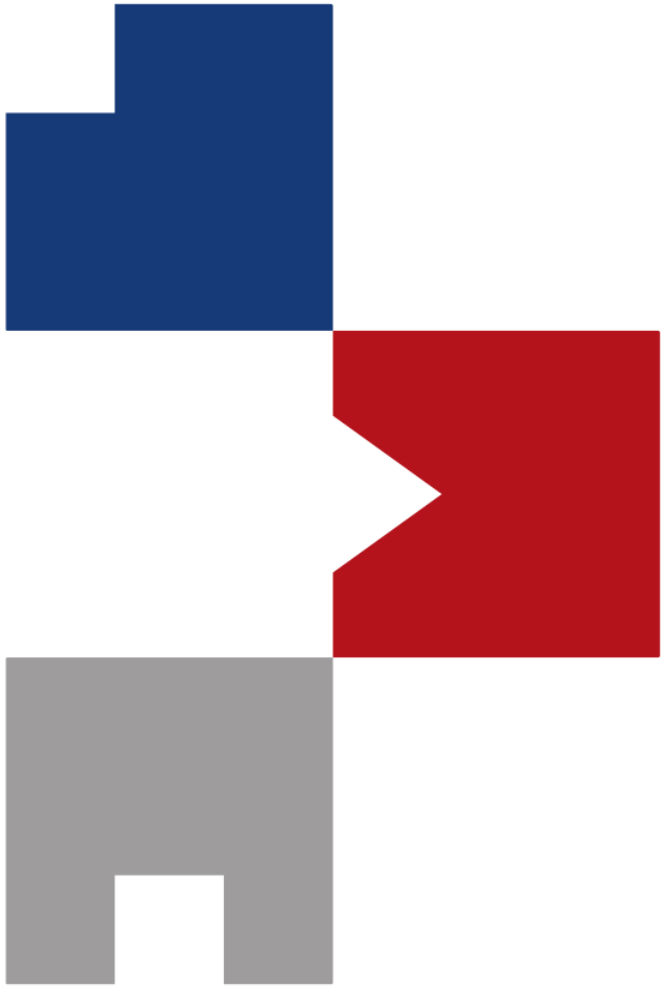
Note) Slides and example codes are available:
https://github.com/mint-lab/3dv_tutorial



An Invitation to 3D Vision: Solving Problems

Sunglok Choi, Assistant Professor, Ph.D.
Dept. of Computer Science and Engineering, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

Table of Contents: Solving Problems

- **Solving Linear Equations using Linear Algebra**
- **Solving Nonlinear Equations using Nonlinear Optimization**
- **Solving Linear Equations in 3D Vision**
  - Affine transformation estimation
  - Planar homography estimation
    - Note) Image warping using homography
  - Fundamental matrix estimation
  - Triangulation
- **Solving Nonlinear Equations in 3D Vision**
  - Absolute camera pose estimation (PnP)
  - Camera calibration

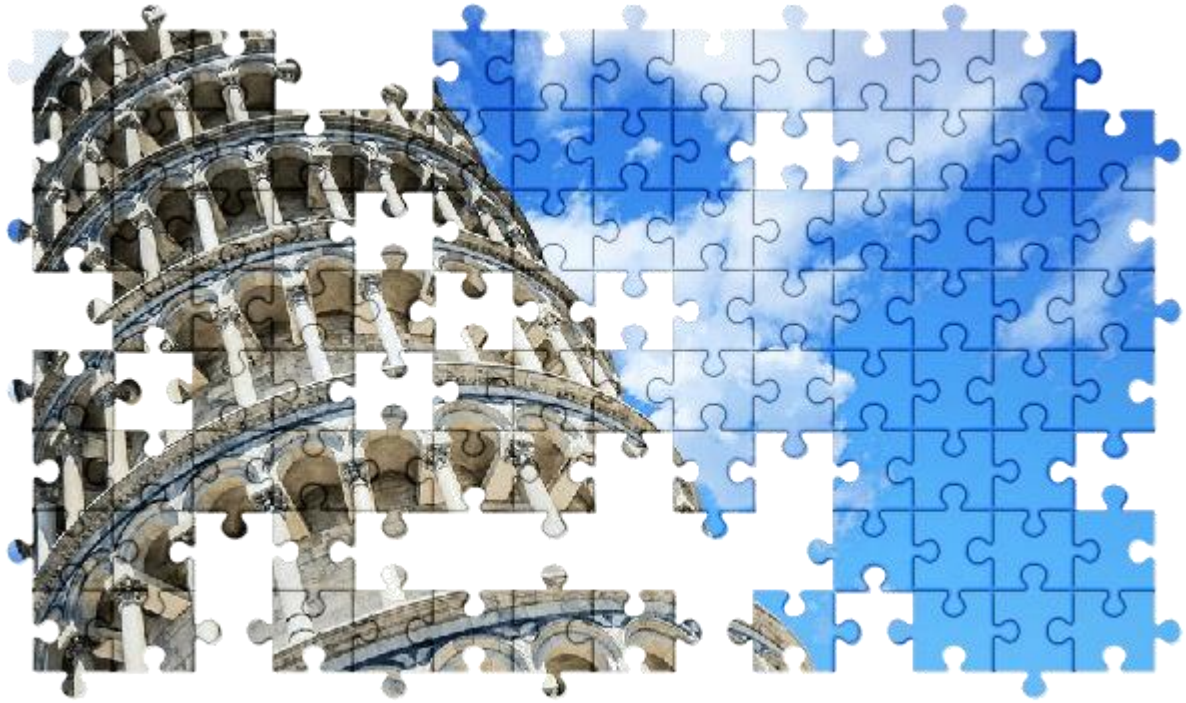


Image: Jigsaw Puzzles Online

Getting Started with a Simple Problem

- Jane bought 3 apples and 4 oranges and paid 10,000 KRW.
- Daniel bought 5 apples and 2 oranges and paid 12,000 KRW.

- **Question) How much are an apple and an orange, respectively?**

- **Answer) Calculate the price of an apple and an orange**
  - Unknown: the price of an apple x and the price of an orange y
  - Constraints: $3x + 4y = 10000$ and $5x + 2y = 12000$
  - Solution using an inverse matrix

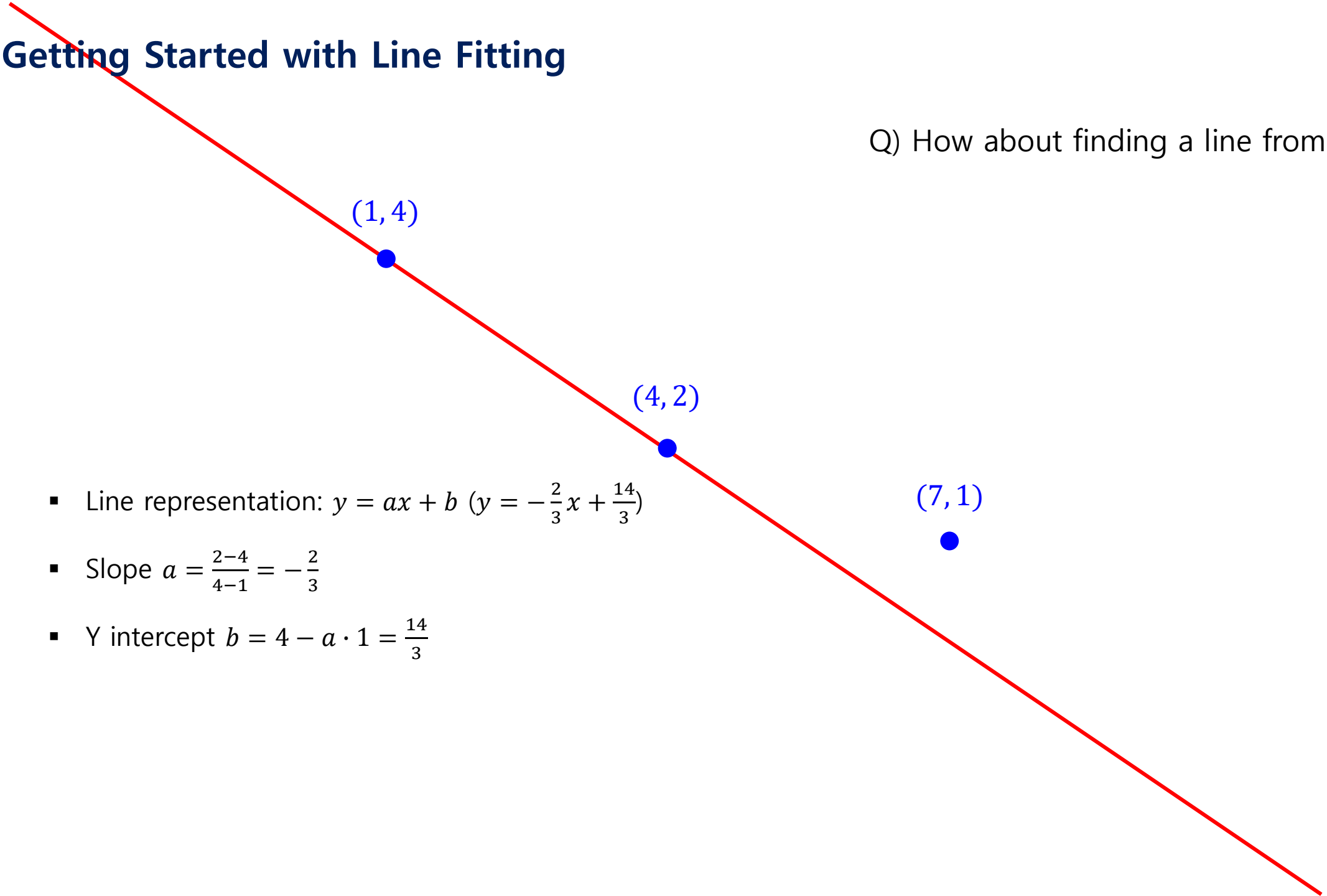
$$\begin{bmatrix} 3 & 4 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 10000 \\ 12000 \end{bmatrix} \rightarrow \mathbf{A}\mathbf{x} = \mathbf{b}$$

Solve $\mathbf{A}\mathbf{x} = \mathbf{b} \rightarrow \mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \begin{bmatrix} 2000 \\ 1000 \end{bmatrix}$

Therefore, an apple is 2,000 KRW, and an orange is 1,000 KRW.

Getting Started with Line Fitting

Q) How about finding a line from three points?



- Line representation: $y = ax + b$ $(y = -\frac{2}{3}x + \frac{14}{3})$
- Slope $a = \frac{2-4}{4-1} = -\frac{2}{3}$
- Y intercept $b = 4 - a \cdot 1 = \frac{14}{3}$

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from two points, $(1, 4)$ and $(4, 2)$** Q) How about finding a line from three points?
  - Unknown: Line parameters a and b (line representation: $y = ax + b$)
  - Constraints: $a \cdot 1 + b = 4$ and $a \cdot 4 + b = 2$
  - Solution

$$\begin{bmatrix} 1 & 1 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{b}$$

$$\therefore \mathbf{x} = \mathrm{A}^{-1}\mathbf{b} = \frac{1}{3}\begin{bmatrix} -2 \\ 14 \end{bmatrix}$$

- **Example) Line fitting from two points, $(1, 4)$ and $(4, 2)$, using NumPy**

```
import numpy as np

A = np.array([[1., 1.], [4., 1.]])
b = np.array([[4.], [2.]])
A_inv = np.linalg.inv(A)
print(A_inv * b) # [[-1.33333333  1.33333333]
                 #  [ 2.66666667 -0.66666667]] Note) broadcast
print(A_inv @ b) # [[-0.66666667]
                 #  [ 4.66666667]] Note) A_inv.matmul(b)
```

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from more than two points such as $(1,4)$, $(4,2)$, and $(7,1)$**
  - Unknown: Line parameters a and b (line representation: $y = ax + b$)
  - Constraints: $a \cdot 1 + b = 4$, $a \cdot 4 + b = 2$, and $a \cdot 7 + b = 1$
  - Solution

$$\begin{bmatrix} 1 & 1 \\ 4 & 1 \\ 7 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{b}$$

$\therefore \mathbf{x} = \mathrm{A}^{\dagger}\mathbf{b}$ where $\mathrm{A}^{\dagger}$ is a pseudo-inverse (Note: $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathrm{A}\mathbf{x} - \mathbf{b}\|_2^2$)

- **Note) Pseudo-inverse** (~ a generalized matrix inverse)
  - **Not necessarily square** (A: $m \times n$ matrix, $\mathrm{A}^{\dagger}$: $n \times m$ matrix)
  - Left inverse ($\mathrm{A}^{\dagger}\mathrm{A} = \mathrm{I}_n$): $\mathrm{A}^{\dagger} = (\mathrm{A}^{\mathsf{T}}\mathrm{A})^{-1}\mathrm{A}^{\mathsf{T}}$
    - If A has linearly independent columns ($\mathrm{rank}(\mathrm{A}) = n$)
  - Right inverse ($\mathrm{A}\mathrm{A}^{\dagger} = \mathrm{I}_m$): $\mathrm{A}^{\mathsf{T}} = \mathrm{A}^{\mathsf{T}}(\mathrm{A}\mathrm{A}^{\mathsf{T}})^{-1}$
    - If A has linearly independent rows ($\mathrm{rank}(\mathrm{A}) = m$)

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from more than two points such as $(1, 4)$, $(4, 2)$, and $(7, 1)$**
  - Unknown: Line parameters a and b (line representation: $y = ax + b$)
  - Constraints: $a \cdot 1 + b = 4$, $a \cdot 4 + b = 2$, and $a \cdot 7 + b = 1$
  - Solution

$$\begin{bmatrix} 1 & 1 \\ 4 & 1 \\ 7 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{b}$$

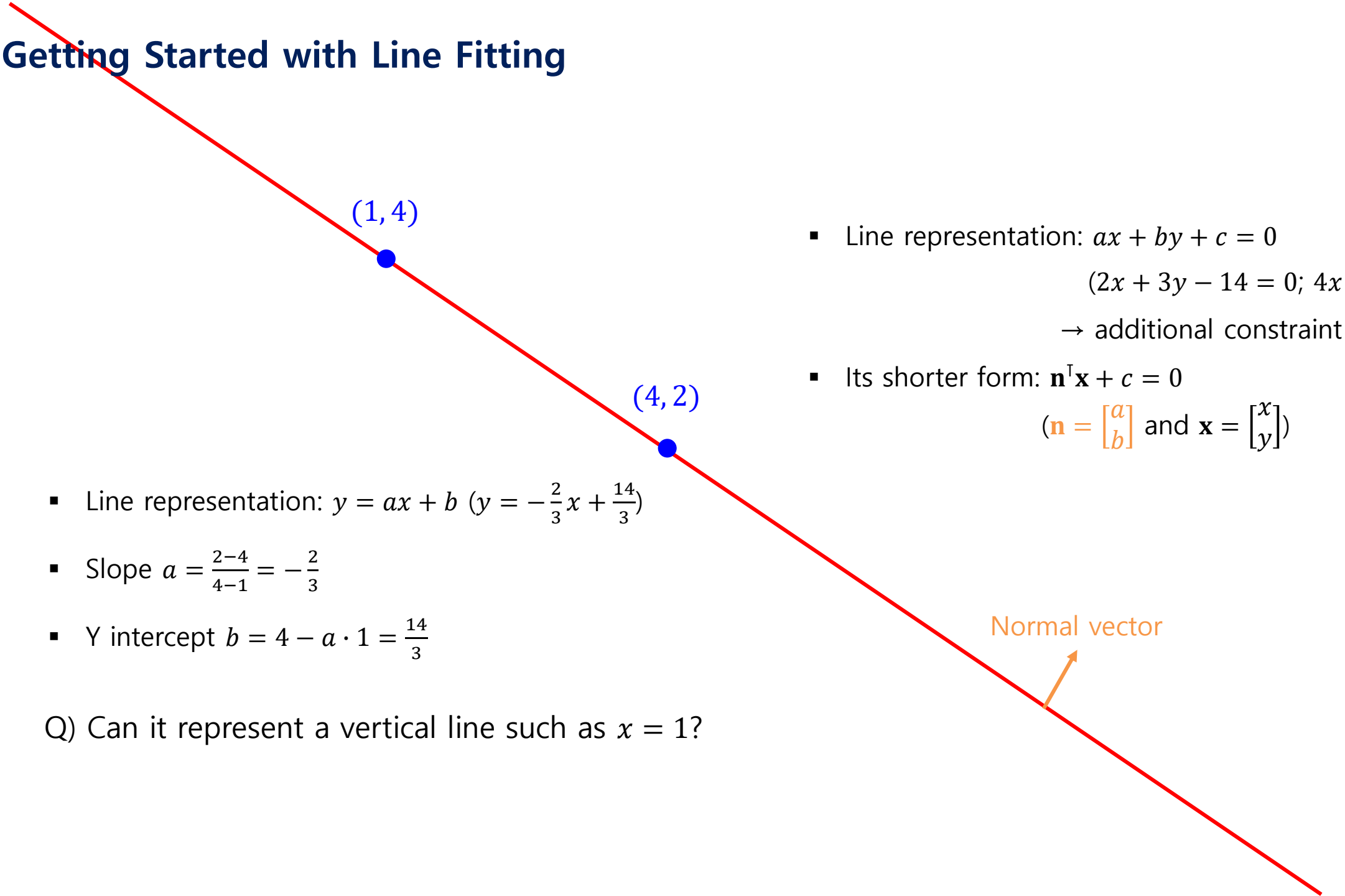
$\therefore \mathbf{x} = \mathrm{A}^{\dagger}\mathbf{b}$ where $\mathrm{A}^{\dagger}$ is a pseudo-inverse (Note: $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathrm{A}\mathbf{x} - \mathbf{b}\|_2^2$)

- **Example) Line fitting from more than two points such as $(1, 4)$, $(4, 2)$, and $(7, 1)$, using NumPy**

```
import numpy as np

A = np.array([[1., 1.], [4., 1.], [7., 1.]])
b = np.array([[4.], [2.], [1.]])
A_inv = np.linalg.pinv(A)
print(A_inv @ b) # [[-0.5], [ 4.33333333]]
```

Getting Started with Line Fitting



- Line representation: $ax + by + c = 0$

  $(2x + 3y - 14 = 0;\ 4x + 6y - 28 = 0)$

  → additional constraint $a^2 + b^2 = 1$

- Its shorter form: $\mathbf{n}^\intercal \mathbf{x} + c = 0$

  ($\mathbf{n} = \begin{bmatrix} a \\ b \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$)

- Line representation: $y = ax + b$ $(y = -\frac{2}{3}x + \frac{14}{3})$
- Slope $a = \frac{2-4}{4-1} = -\frac{2}{3}$
- Y intercept $b = 4 - a \cdot 1 = \frac{14}{3}$

Q) Can it represent a vertical line such as $x = 1$?

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from two points, $(1, 4)$ and $(4, 2)$**
  - Unknown: Line parameters a, b, and c (line representation: $ax + by + c = 0$)
  - Constraints: $a \cdot 1 + b \cdot 4 + c = 0$ and $a \cdot 4 + b \cdot 2 + c = 0$
  - Solution

$$\begin{bmatrix} 1 & 4 & 1 \\ 4 & 2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{0} \text{ (homogeneous equation)} \quad \sout{\mathbf{x} = \mathrm{A}^{\pm}\mathbf{b}}$$

$$\therefore \mathbf{x} = \mathrm{null}(\mathrm{A}) = [2, 3, -14]^{\mathrm{T}} \rightarrow 2x + 3y - 14 = 0$$

- **Note) Null space** (a.k.a. kernel)
  - A set of vectors which map A (m-by-n matrix) to the zero vector

$$\mathrm{null}(\mathrm{A}) = \{ \mathbf{v} \in K^n \mid \mathrm{A}\mathbf{v} = \mathbf{0} \}$$

  - Rank-nullity theorem: $\mathrm{rank}(\mathrm{A}) + \mathrm{nullity}(\mathrm{A}) = n$

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from two points, $(1, 4)$ and $(4, 2)$**
  - Unknown: Line parameters a, b, and c (line representation: $ax + by + c = 0$)
  - Constraints: $a \cdot 1 + b \cdot 4 + c = 0$ and $a \cdot 4 + b \cdot 2 + c = 0$
  - Solution

$$\begin{bmatrix} 1 & 4 & 1 \\ 4 & 2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{0} \text{ (homogeneous equation)} \quad \sout{\mathbf{x} = \mathrm{A}^{\pm}\mathbf{b}}$$

$$\therefore \mathbf{x} = \mathrm{null}(\mathrm{A}) = [2, 3, -14]^{\mathrm{T}} \rightarrow 2x + 3y - 14 = 0$$

- **Example) Line fitting from two points, $(1, 4)$ and $(4, 2)$, using NumPy**

```
import numpy as np
from scipy import linalg

A = np.array([[1., 4., 1.], [4., 2., 1.]])
x = linalg.null_space(A)
print(x / x[0]) # [[1.], [1.5], [-7.]]
```

Q) How about finding a line from three points?

A) No null space for $\mathrm{A} = \begin{bmatrix} 1 & 4 & 1 \\ 4 & 2 & 1 \\ 7 & 1 & 1 \end{bmatrix}$ ($\because$ full rank)

Solving Linear Equations using Linear Algebra

- **Example) Line fitting from more than two points such as $(1, 4)$, $(4, 2)$, and $(7, 1)$**
  - Unknown: Line parameters a, b, and c (line representation: $ax + by + c = 0$)
  - Constraints: $a \cdot 1 + b \cdot 4 + c = 0$, $a \cdot 4 + b \cdot 2 + c = 0$, and $a \cdot 7 + b \cdot 1 + c = 0$
  - Solution

$$A = \begin{bmatrix} 1 & 4 & 1 \\ 4 & 2 & 1 \\ 7 & 1 & 1 \end{bmatrix} = USV^T$$ using singular value decomposition (SVD)

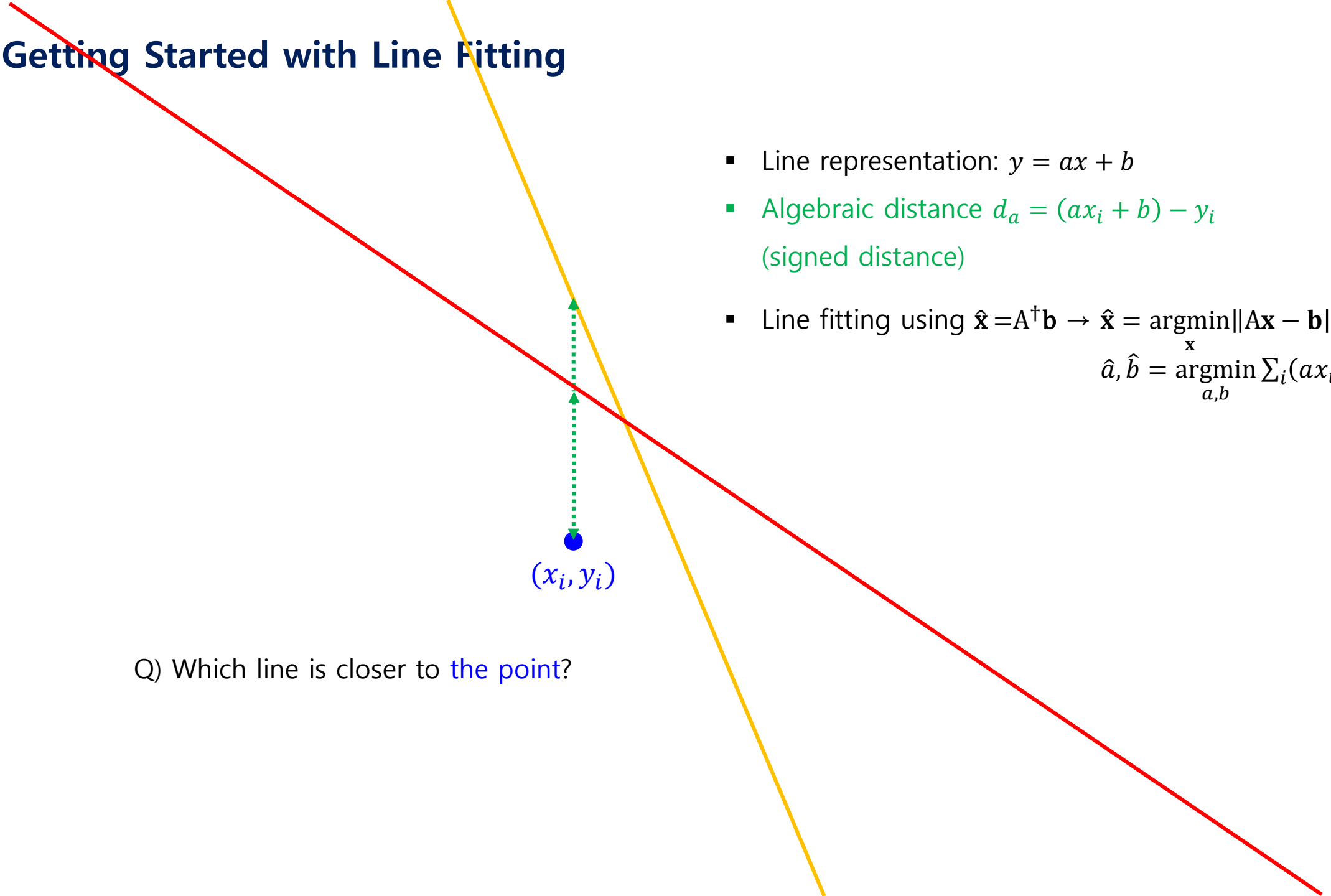
$\therefore \mathbf{x}$ is the last row of V^T. (Note: $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|A\mathbf{x}\|^2$ with $\|\mathbf{x}\|^2 = 1$ condition)

- **Example) Line fitting from more than two points such as $(1, 4)$, $(4, 2)$, and $(7, 1)$, using NumPy**

```
import numpy as np

A = np.array([[1., 4., 1.], [4., 2., 1.], [7., 1., 1.]])
U, S, Vt = np.linalg.svd(A, full_matrices=True)
x = Vt[-1].T
print(x / -x[1]) # [[-0.50824973], [-1.], [4.37835787]]
                 # Note) [[a], [b]] = [[-0.5], [ 4.33333333]]
```

Getting Started with Line Fitting

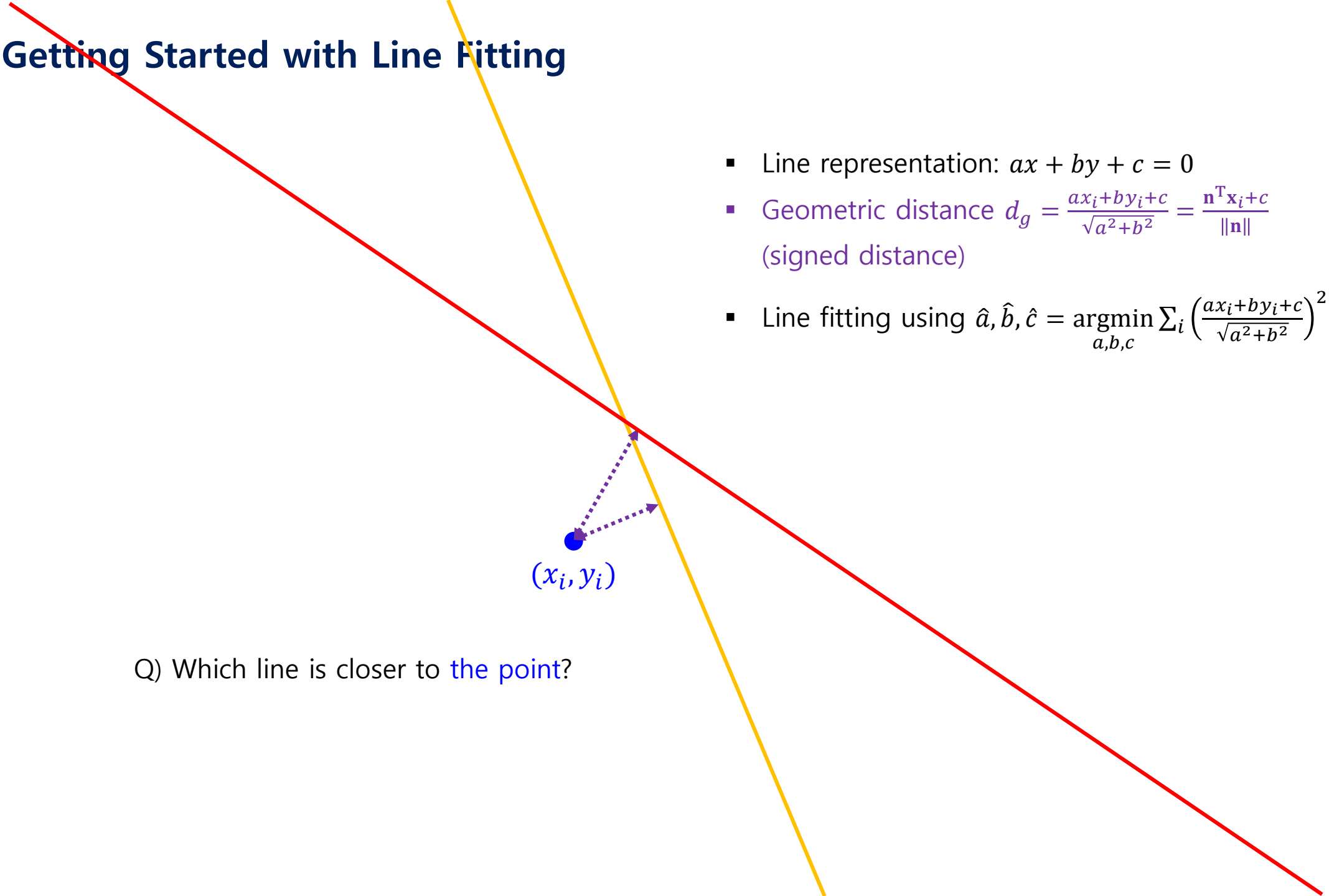


- Line representation: $y = ax + b$
- Algebraic distance $d_a = (ax_i + b) - y_i$
  (signed distance)
- Line fitting using $\hat{\mathbf{x}} = \mathrm{A}^{\dagger}\mathbf{b} \rightarrow \hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathrm{A}\mathbf{x} - \mathbf{b}\|_2^2$

$$\hat{a}, \hat{b} = \underset{a,b}{\operatorname{argmin}} \sum_i (ax_i + b - y_i)^2$$

Q) Which line is closer to the point?

Getting Started with Line Fitting



- Line representation: $ax + by + c = 0$
- Geometric distance $d_g = \frac{ax_i+by_i+c}{\sqrt{a^2+b^2}} = \frac{\mathbf{n}^T\mathbf{x}_i+c}{\|\mathbf{n}\|}$ (signed distance)
- Line fitting using $\hat{a}, \hat{b}, \hat{c} = \underset{a,b,c}{\operatorname{argmin}} \sum_i \left(\frac{ax_i+by_i+c}{\sqrt{a^2+b^2}}\right)^2$

Q) Which line is closer to the point?

Solving Nonlinear Equation using Nonlinear Optimization

- **Nonlinear optimization** is the process of solving an optimization problem where some of the constraints or the objective function are **nonlinear**.
  - Alias: Nonlinear programming (NLP)
  - Mathematically, $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \leq 0$ for each $i \in \{1, \ldots, m\}$
    $h_j(\mathbf{x}) = 0$ for each $j \in \{1, \ldots, p\}$
    $\mathbf{x} \in \mathrm{X}$ (X is a subset of $\mathbb{R}^n$)
    - $f(\mathbf{x})$: The real-valued objective function
    - $g_i(\mathbf{x})$: The i-th real-valued inequality constraint function
    - $h_j(\mathbf{x})$: The j-th real-valued equality constraint function
  - Example) The objective function $f(x, y) = 4 - (x^2 + y^2)$ is nonlinear.

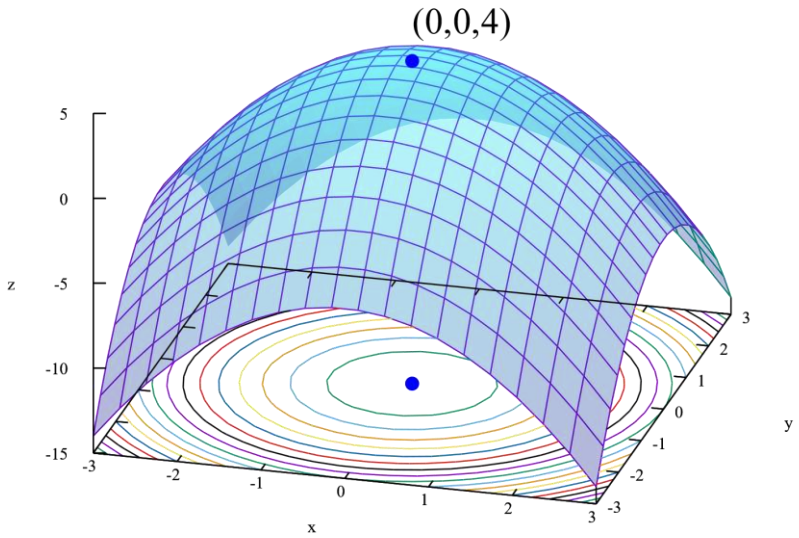
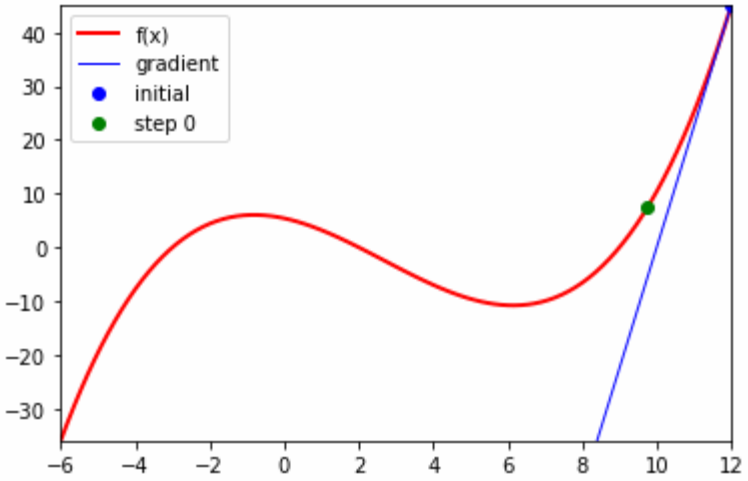


Image: Wikipedia (Mathematical optimization)

Nonlinear Optimization

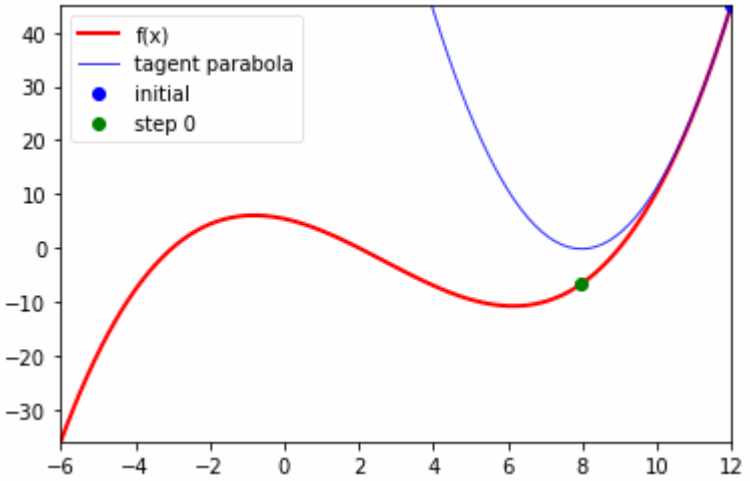
- **Gradient descent**
  - A **first-order iterative algorithm** for finding a local minimum of a differentiable function by moving to the opposite direction of the gradient of the function at the current point
  - Mathematically, $x_{t+1} = x_t - \gamma f'(x_t)$
    - γ: The step size (a.k.a. learning rate)



- Note) **Stochastic gradient descent (SGD)**
  - SGD uses an approximate gradient (calculated from a randomly selected subset of the given data) instead of the actual gradient (calculated from the entire data).
  - SGD variants: AdaGrad, RMSProp, Adam, …

Nonlinear Optimization

- **Newton's method**
  - A **second-order iterative algorithm** for finding a local minimum of a differentiable function by finding the minima of the locally approximated parabola of the function at the current point
  - Mathematically, $x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$
    - The step size is **not** required.



- Note) **Gauss-Newton method**
  - A special case for non-linear least squares problems
    - When the function has a form of $f(x) = r^2(x)$,
    - Newton's method becomes $x_{t+1} = x_t - \frac{r(x_t)}{r'(x_t)}$ (without the 2nd-order derivative)

Solving Nonlinear Equation using Nonlinear Optimization

- **Example) Line fitting from more than two points such as $(1,4)$, $(4,2)$, and $(7,1)$, using SciPy**
  - Unknown: Line parameters a, b, and c (line representation: $ax + by + c = 0$)
  - Cost function: $f(a,b,c) = \sum_i \left(\frac{ax_i+by_i+c}{\sqrt{a^2+b^2}}\right)^2$
  - Optimizer: Gauss-Newton method (least squares)

```
import numpy as np
from scipy.optimize import least_squares

def geometric_error(line, pts):
    a, b, c = line
    err = [(a*x + b*y + c) / np.sqrt(a*a + b*b) for (x, y) in pts]
    return err

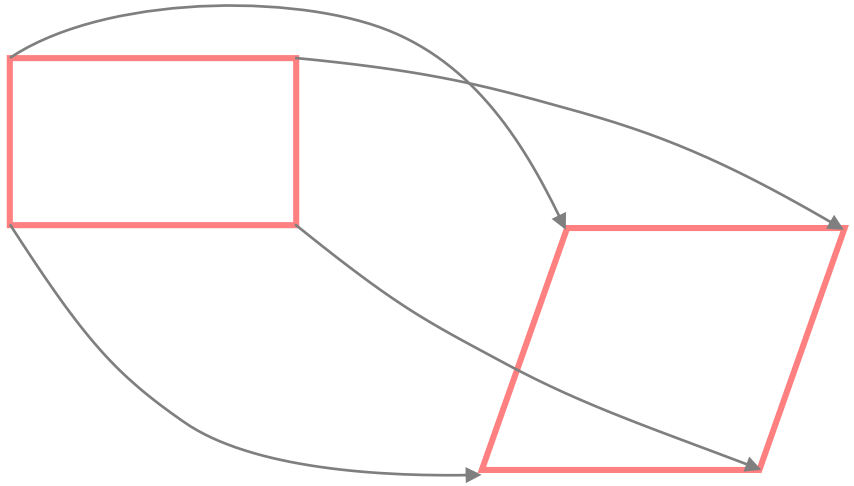
pts = [(1, 4), (4, 2), (7, 1)]
line_init = [1, 1, 0]
result = least_squares(geometric_error, line_init, args=(pts,))
line = result['x'] / -result['x'][1] # [-0.50372575, -1., 4.34823633]
```

Summary) Solving Line Fitting Problems

- **Linear equations**
  - Inhomogeneous equations $\mathrm{A}\mathbf{x} = \mathbf{b} \rightarrow \mathbf{x} = \mathrm{A}^{\dagger}\mathbf{b}$ where $\mathrm{A}^{\dagger}$ is a pseudo-inverse
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathrm{A}\mathbf{x} - \mathbf{b}\|_2^2$
  - Homogeneous equation $\mathrm{A}\mathbf{x} = \mathbf{0} \rightarrow \mathbf{x}$ is the last row of $\mathrm{V}^{\intercal}$ where $\mathrm{A} = \mathrm{U}\mathrm{S}\mathrm{V}^{\intercal}$ (from singular value decomposition)
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathrm{A}\mathbf{x}\|^2$ with $\|\mathbf{x}\|^2 = 1$ condition
- **Nonlinear equations**
  - Nonlinear optimization: ~~Gradient descent, Newton's method, and Gauss-Newton method (for $f(x) = r^2(x)$)~~
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} f(\mathbf{x})$
    - Note) Let's use magic tools such as scipy.optimize and Ceres Solver for better performance.

Solving Linear Equations in 3D Vision

- **Affine transformation estimation**
  - Unknown: Affine transformation H (6 DOF)
  - Given: Point correspondence $(\mathbf{x}_1, \mathbf{x}'_1)$, …, $(\mathbf{x}_n, \mathbf{x}'_n)$
  - Constraints: n x affine transformation $\mathbf{x}'_i = \mathrm{H}\mathbf{x}_i$
  - Solutions ($n \geq 3$)
    - OpenCV: `cv.getAffineTransform()`
    - **Affine transformation estimation**
      - Affine transformation: $\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix}$
      - For n pairs of points, $\begin{bmatrix} x_1 & y_1 & 0 & 0 & 1 & 0 \\ 0 & 0 & x_1 & y_1 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 0 & 0 & 1 & 0 \\ 0 & 0 & x_n & y_n & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{12} \\ a_{21} \\ a_{22} \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix} \rightarrow \mathrm{A}\mathbf{x} = \mathbf{b}$
      - Solve $\mathrm{A}\mathbf{x} = \mathbf{b}$ and organize H from $\mathbf{x}$



Solving Linear Equations in 3D Vision

- **Affine transformation estimation** [affine_estimation_implement.py]

```
import numpy as np
import cv2 as cv

def getAffineTransform(src, dst):
    if len(src) == len(dst):
        # Solve 'Ax = b'
        A, b = [], []
        for p, q in zip(src, dst):
            A.append([p[0], p[1], 0, 0, 1, 0])
            A.append([0, 0, p[0], p[1], 0, 1])
            b.append(q[0])
            b.append(q[1])
        x = np.linalg.pinv(A) @ b

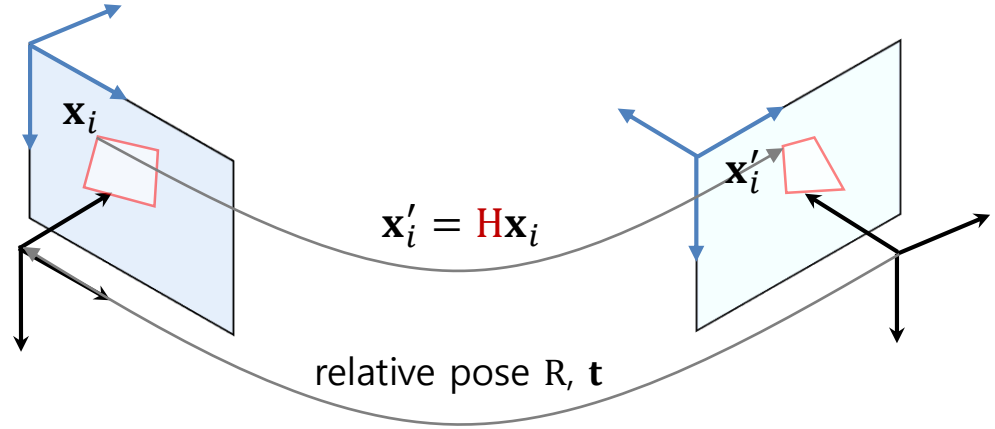
        # Reorganize `x` as a matrix
        H = np.array([[x[0], x[1], x[4]], [x[2], x[3], x[5]]])
        return H

if __name__ == '__main__':
    src = np.array([[115, 401], [776, 180], [330, 793]], dtype=np.float32)
    dst = np.array([[0, 0], [900, 0], [0, 500]], dtype=np.float32)

    my_H = getAffineTransform(src, dst)
    cv_H = cv.getAffineTransform(src, dst) # Note) It accepts only 3 pairs of points.
    ...
```

$$\begin{bmatrix} x_1 & y_1 & 0 & 0 & 1 & 0 \\ 0 & 0 & x_1 & y_1 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 0 & 0 & 1 & 0 \\ 0 & 0 & x_n & y_n & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{12} \\ a_{21} \\ a_{22} \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x_1' \\ y_1' \\ \vdots \\ x_n' \\ y_n' \end{bmatrix}$$

Solving Linear Equations in 3D Vision



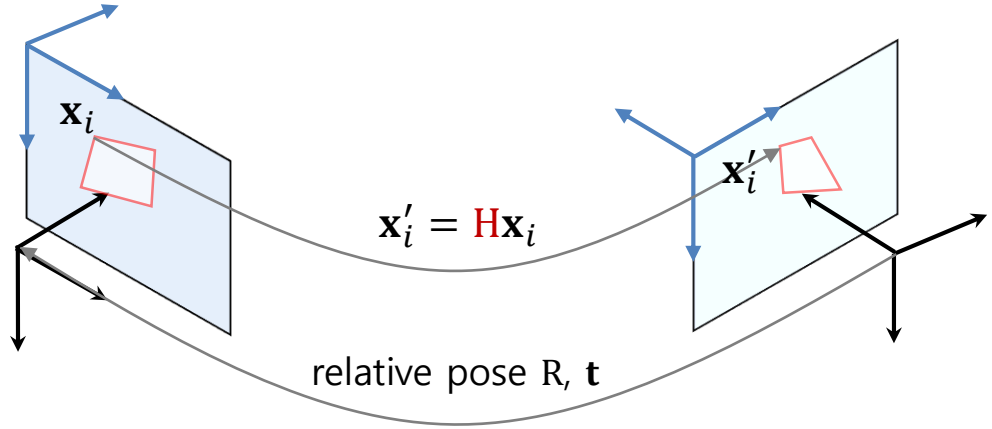
- **Planar homography estimation**
  - Unknown: Planar homography H (8 DOF ← up to scale)
  - Given: Point correspondence $(\mathbf{x}_1, \mathbf{x}_1'), \ldots, (\mathbf{x}_n, \mathbf{x}_n')$
  - ~~Constraints: n x projective transformation $\mathbf{x}_i' = \mathrm{H}\mathbf{x}_i$~~

$$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ v_1 & v_2 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} u_i \\ v_i \\ w_i \end{bmatrix} \rightarrow \mathbf{x}_i' = \begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} = \frac{1}{w_i} \begin{bmatrix} u_i \\ v_i \\ w_i \end{bmatrix}$$

  - Modified constraints: n x transformation $\mathbf{x}_i' \sim \mathrm{H}\mathbf{x}_i$ (similarity) or $\mathbf{x}_i' = \lambda_i \mathrm{H}\mathbf{x}_i$ or $\mathbf{x}_i' \times (\mathrm{H}\mathbf{x}_i) = \mathbf{0}$
    - Note) The last element of $\mathbf{x}_i$ and $\mathbf{x}_i'$ in homogeneous coordinates are not necessary to be 1.
  - Solutions ($n \geq 4$) → 4-point algorithm
    - OpenCV: `cv.getPerspectiveTransform()` and `cv.findHomography()`

Solving Linear Equations in 3D Vision

- **Planar homography estimation**
  - Unknown: Planar homography H (8 DOF ← up to scale)
  - Given: Point correspondence $(\mathbf{x}_1, \mathbf{x}_1'), \ldots, (\mathbf{x}_n, \mathbf{x}_n')$
  - Constraints: n x transformation $\mathbf{x}_i' \times (\mathrm{H}\mathbf{x}_i) = \mathbf{0}$
  - Solutions ($n \geq 4$) → 4-point algorithm
    - OpenCV: `cv.getPerspectiveTransform()` and `cv.findHomography()`
    - **4-point algorithm**



$$[x_i' \quad y_i' \quad w_i'] \times \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ w_i \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \rightarrow \begin{matrix} w_i'(x_i h_{21} + y_i h_{22} + w_i h_{23}) - y_i'(x_i h_{31} + y_i h_{32} + w_i h_{33}) = 0 \\ w_i'(x_i h_{11} + y_i h_{12} + w_i h_{13}) - x_i'(x_i h_{31} + y_i h_{32} + w_i h_{33}) = 0 \\ y_i'(x_i h_{11} + y_i h_{12} + w_i h_{13}) - x_i'(x_i h_{21} + y_i h_{22} + w_i h_{23}) = 0 \end{matrix}$$

  - For n pairs, $\begin{bmatrix} 0 & 0 & 0 & w_1'x_1 & w_1'y_1 & w_1'w_1 & -y_1'x_1 & -y_1'y_1 & -y_1'w_1 \\ w_1'x_1 & w_1'y_1 & w_1'w_1 & 0 & 0 & 0 & -x_1'x_1 & -x_1'y_1 & -x_1'w_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & w_n'x_n & w_n'y_n & w_n'w_n & -y_n'x_n & -y_n'y_n & -y_n'w_n \\ w_n'x_n & w_n'y_n & w_n'w_n & 0 & 0 & 0 & -x_n'x_n & -x_n'y_n & -x_n'w_n \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = 0 \rightarrow \mathrm{A}\mathbf{x} = 0$
  - Solve $\mathrm{A}\mathbf{x} = 0$ and reorganize H (3x3 matrix)

Solving Linear Equations in 3D Vision

- **Planar homography estimation** [homography_estimation_implement.py]

```python
import numpy as np
import cv2 as cv

def getPerspectiveTransform(src, dst):
    if len(src) == len(dst):
        # Make homogeneous coordinates if necessary
        if src.shape[1] == 2:
            src = np.hstack((src, np.ones((len(src), 1), dtype=src.dtype)))
        if dst.shape[1] == 2:
            dst = np.hstack((dst, np.ones((len(dst), 1), dtype=dst.dtype)))

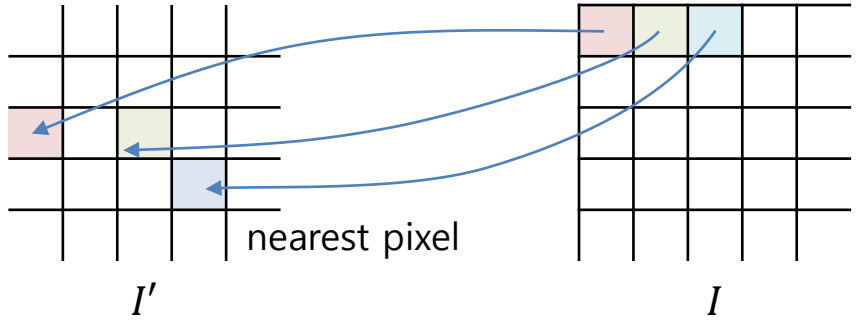
        # Solve 'Ax = 0'
        A = []
        for p, q in zip(src, dst):
            A.append([0, 0, 0, q[2]*p[0], q[2]*p[1], q[2]*p[2], -q[1]*p[0], -q[1]*p[1], -q[1]*p[2]])
            A.append([q[2]*p[0], q[2]*p[1], q[2]*p[2], 0, 0, 0, -q[0]*p[0], -q[0]*p[1], -q[0]*p[2]])
        _, _, Vt = np.linalg.svd(A, full_matrices=True)
        x = Vt[-1]

        # Reorganize `x` as a matrix
        H = x.reshape(3, -1) / x[-1] # Normalize the last element as 1
        return H

if __name__ == '__main__':
    ...
```

Appendix) Image Warping

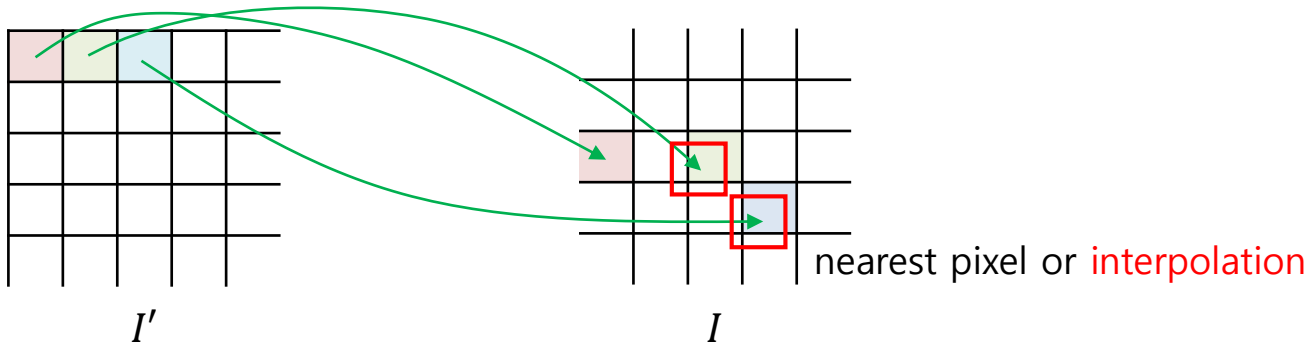
- **Example) Image warping using homography**
  - Target: Transformed image I'
  - Source: Original image I
  - Relationship: $I'(\mathbf{x}') = I(\mathbf{x})$ where $\mathbf{x}' = \mathrm{H}\mathbf{x}$
  - Method #1) Select a pair of points, $\mathbf{x} \in \{(0,0),(0,1),\dots,(1,0),(1,1),\dots\}$ and $\mathbf{x}' = \mathrm{H}\mathbf{x}$







  - Method #2) Select a pair of points, $\mathbf{x}' \in \{(0,0),(0,1),\dots,(1,0),(1,1),\dots\}$ and $\mathbf{x} = \mathrm{H}^{-1}\mathbf{x}'$





- **Example) Image warping using homography** [image_warping_implement.py]

```
import cv2 as cv
...

def warpPerspective1(src, H, dst_size):
    # Generate an empty image
    width, height = dst_size
    channel = src.shape[2] if src.ndim > 2 else 1
    dst = np.zeros((height, width, channel), dtype=src.dtype)

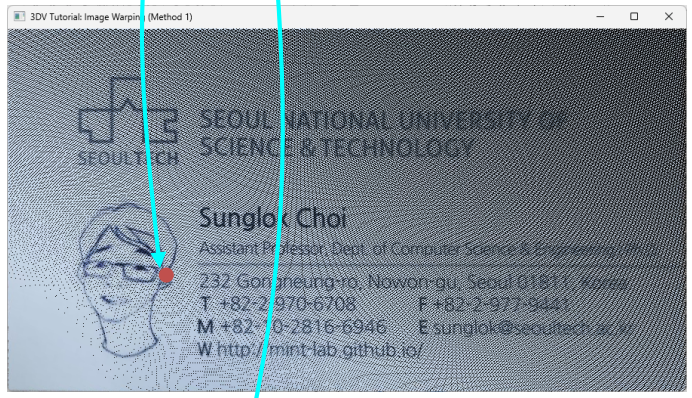
    # Copy a pixel from `src` to `dst` (forward mapping)
    for py in range(img.shape[0]):
        for px in range(img.shape[1]):
            q = H @ [px, py, 1]
            qx, qy = int(q[0]/q[-1] + 0.5), int(q[1]/q[-1] + 0.5)
            if qx >= 0 and qy >= 0 and qx < width and qy < height:
                dst[qy, qx] = src[py, px]
    return dst

def warpPerspective2(src, H, dst_size):
    # Generate an empty image
    width, height = dst_size
    channel = src.shape[2] if src.ndim > 2 else 1
    dst = np.zeros((height, width, channel), dtype=src.dtype)

    # Copy a pixel from `src` to `dst` (backward mapping)
    H_inv = np.linalg.inv(H)
    for qy in range(height):
        for qx in range(width):
            p = H_inv @ [qx, qy, 1]
            px, py = int(p[0]/p[-1] + 0.5), int(p[1]/p[-1] + 0.5)
            if px >= 0 and py >= 0 and px < src.shape[1] and py < src.shape[0]:
                dst[qy, qx] = src[py, px]
    return dst
```



$\mathbf{x}' = \mathrm{H}\mathbf{x}$

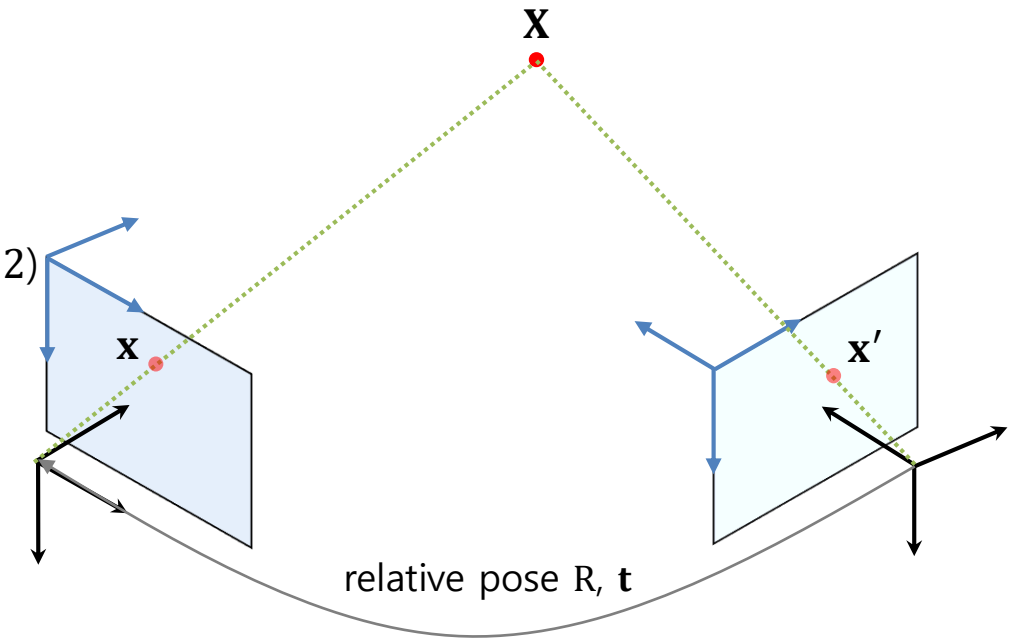


$\mathbf{x} = \mathrm{H}^{-1}\mathbf{x}'$



Solving Linear Equations in 3D Vision

- **Fundamental matrix estimation**
  - Unknown: Fundamental matrix F (7 DOF ← up to scale, $\mathrm{rank(F)} = 2$)
  - Given: Point correspondence $(\mathbf{x}_1, \mathbf{x}'_1)$, …, $(\mathbf{x}_n, \mathbf{x}'_n)$
  - Constraints: n x epipolar constraint $\mathbf{x}'^{\mathrm{T}}\mathrm{F}\mathbf{x} = 0$
  - Solutions ($n \geq 7$) → 7-point and 8-point algorithms
    - OpenCV: `cv.findFundamentalMat()`
    - **8-point algorithm**
      - Epipolar constraint: $\begin{bmatrix} x'_i & y'_i & w'_i \end{bmatrix} \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ w_i \end{bmatrix} = 0$
      - For n pairs of points, $\begin{bmatrix} x'_1x_1 & x'_1y_1 & x'_1w_1 & y'_1x_1 & y'_1y_1 & y'_1w_1 & w'_1x_1 & w'_1y_1 & w'_1w_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x'_nx_n & x'_ny_n & x'_nw_n & y'_nx_n & y'_ny_n & y'_nw_n & w'_nx_n & w'_ny_n & w'_nw_n \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0 \rightarrow \mathrm{A}\mathbf{x} = 0$
      - Solve $\mathrm{A}\mathbf{x} = 0$ and reorganize F (3x3 matrix)
      - Enforce $\mathrm{rank(F)} = 2$ using singular value decomposition (SVD)



- **Fundamental matrix estimation** [fundamental_mat_estimation_implement.py]

```
import numpy as np
import cv2 as cv

def findFundamentalMat(pts1, pts2):
    if len(pts1) == len(pts2):
        # Make homogeneous coordiates if necessary
        if pts1.shape[1] == 2:
            pts1 = np.hstack((pts1, np.ones((len(pts1), 1), dtype=pts1.dtype)))
        if pts2.shape[1] == 2:
            pts2 = np.hstack((pts2, np.ones((len(pts2), 1), dtype=pts2.dtype)))

        # Solve 'Ax = 0'
        A = []
        for p, q in zip(pts1, pts2):
            A.append([q[0]*p[0], q[0]*p[1], q[0]*p[2], q[1]*p[0], q[1]*p[1], q[1]*p[2], q[2]*p[0], q[2]*p[1], q[2]*p
        _, _, Vt = np.linalg.svd(A, full_matrices=True)
        x = Vt[-1]

        # Reorganize `x` as `F` and enforce 'rank(F) = 2'
        F = x.reshape(3, -1)
        U, S, Vt = np.linalg.svd(F)
        S[-1] = 0
        F = U @ np.diag(S) @ Vt
        return F / F[-1,-1] # Normalize the last element as 1

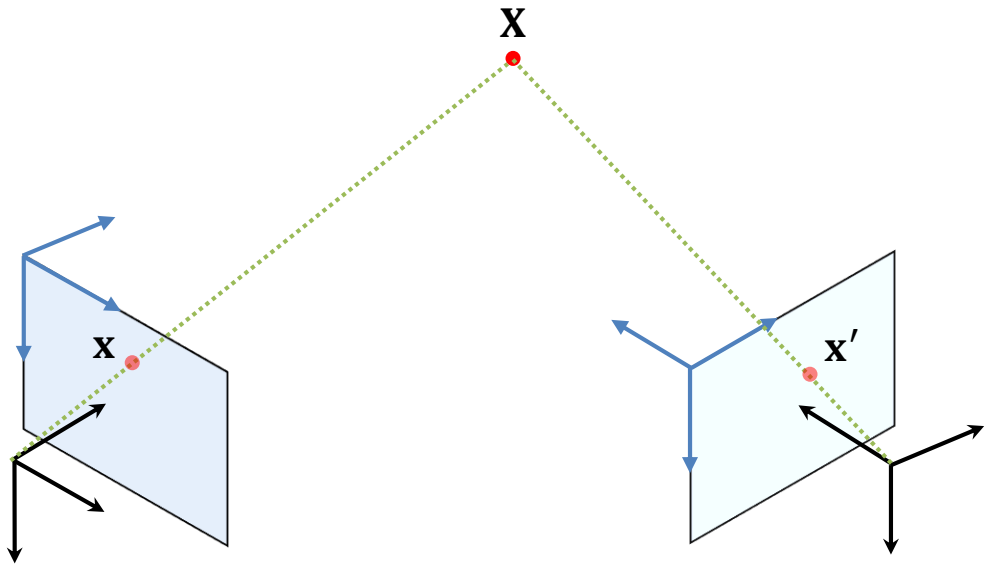
if __name__ == '__main__':
    ...
```

Solving Linear Equations in 3D Vision

- **Triangulation** (point localization)
  - Unknown: Position of a 3D point $\mathbf{X}$ (3 DOF)
  - Given: Point correspondence $(\mathbf{x}, \mathbf{x}')$ and projection matrices $(\mathrm{P}, \mathrm{P}')$
  - Constraints: $\mathbf{x} = \mathrm{K}\,[\,\mathrm{I} \mid \mathbf{0}\,]\,\mathbf{X} = \mathrm{P}\,\mathbf{X}$, $\mathbf{x}' = \mathrm{K}'\,[\,\mathrm{R} \mid \mathbf{t}\,]\,\mathbf{X} = \mathrm{P}'\,\mathbf{X}$
  - Solutions
    - OpenCV: `cv.triangulatePoints()`
    - **Linear triangulation**
      - Projection: $\mathbf{x} = \mathrm{P}\mathbf{X} \rightarrow \mathbf{x} \times (\mathrm{P}\mathbf{X}) = \mathbf{0} \rightarrow$

$$\begin{matrix} x(\mathbf{p}_3^\intercal \mathbf{X}) - w(\mathbf{p}_1^\intercal \mathbf{X}) = 0 \\ y(\mathbf{p}_3^\intercal \mathbf{X}) - w(\mathbf{p}_2^\intercal \mathbf{X}) = 0 \\ x(\mathbf{p}_2^\intercal \mathbf{X}) - y(\mathbf{p}_1^\intercal \mathbf{X}) = 0 \end{matrix} \quad \text{where } \mathrm{P} = \begin{bmatrix} \mathbf{p}_1^\intercal \\ \mathbf{p}_2^\intercal \\ \mathbf{p}_3^\intercal \end{bmatrix}$$

      - Solve $\mathrm{A}\mathbf{X} = 0$ where $\mathrm{A} = \begin{bmatrix} x\mathbf{p}_3^\intercal - w\mathbf{p}_1^\intercal \\ y\mathbf{p}_3^\intercal - w\mathbf{p}_2^\intercal \\ x'\mathbf{p}'^\intercal_3 - w'\mathbf{p}'^\intercal_1 \\ y'\mathbf{p}'^\intercal_3 - w'\mathbf{p}'^\intercal_2 \end{bmatrix}$



- **Triangulation** (point localization) [triangulation_implement.py]

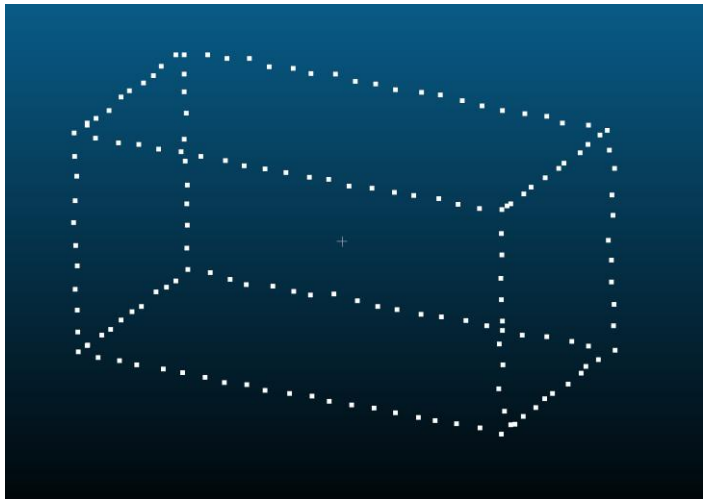
```python
def triangulatePoints(P0, P1, pts0, pts1):
    Xs = []
    for (p, q) in zip(pts0.T, pts1.T):
        # Solve 'AX = 0'
        A = np.vstack((p[0] * P0[2] - P0[0],
                       p[1] * P0[2] - P0[1],
                       q[0] * P1[2] - P1[0],
                       q[1] * P1[2] - P1[1]))
        _, _, Vt = np.linalg.svd(A, full_matrices=True)
        Xs.append(Vt[-1])
    return np.vstack(Xs).T

if __name__ == '__main__':
    f, cx, cy = 1000., 320., 240.
    pts0 = np.loadtxt('../data/image_formation0.xyz')[:,:2]
    pts1 = np.loadtxt('../data/image_formation1.xyz')[:,:2]
    output_file = 'triangulation_implement.xyz'

    # Estimate the relative pose of two views
    F, _ = cv.findFundamentalMat(pts0, pts1, cv.FM_8POINT)
    K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
    E = K.T @ F @ K
    _, R, t, _ = cv.recoverPose(E, pts0, pts1, K)

    # Reconstruct 3D points (triangulation)
    P0 = K @ np.eye(3, 4, dtype=np.float32)
    Rt = np.hstack((R, t))
    P1 = K @ Rt
    X = triangulatePoints(P0, P1, pts0.T, pts1.T)
    X /= X[3]
```

$$\mathrm{A} = \begin{bmatrix} x\mathbf{p}_3^{\mathrm{T}} - w\mathbf{p}_1^{\mathrm{T}} \\ y\mathbf{p}_3^{\mathrm{T}} - w\mathbf{p}_2^{\mathrm{T}} \\ x'\mathbf{p}'^{\mathrm{T}}_3 - w'\mathbf{p}'^{\mathrm{T}}_1 \\ y'\mathbf{p}'^{\mathrm{T}}_3 - w'\mathbf{p}'^{\mathrm{T}}_2 \end{bmatrix}$$



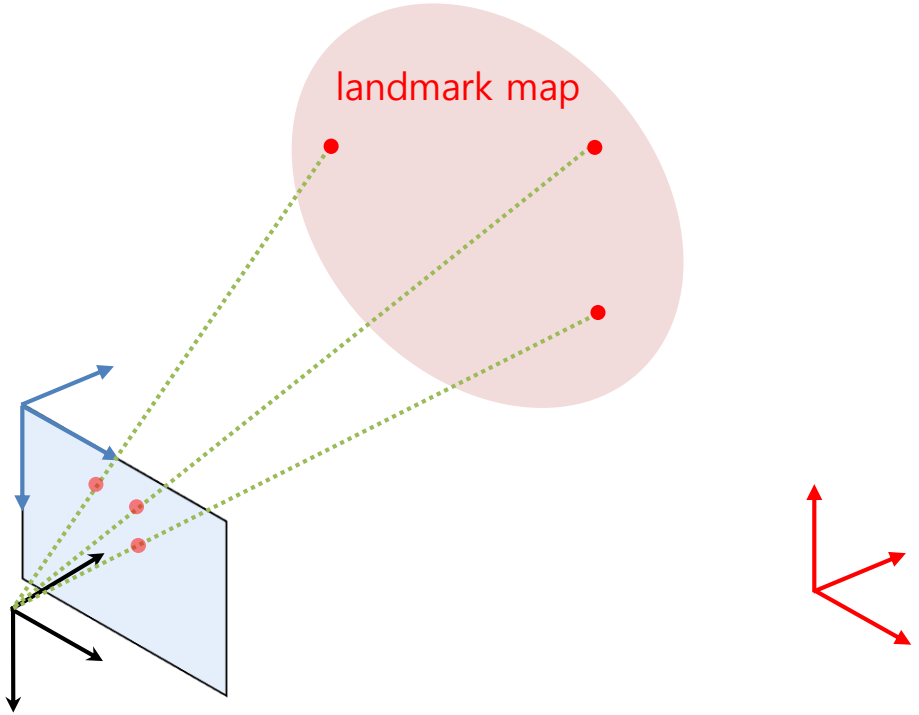
triangulation_implement.xyz

Solving Nonlinear Equations in 3D Vision

- **Absolute camera pose estimation** (perspective-n-point; PnP)
  - Unknown: Camera pose R and $\mathbf{t}$ (6 DOF)
  - Given: 3D points $\mathbf{X}_1$, $\mathbf{X}_2$, …, $\mathbf{X}_n$, their projected points $\mathbf{x}_1$, $\mathbf{x}_2$, …, $\mathbf{x}_n$, and camera matrix K
  - ~~Constraints: n x projection $\mathbf{x}_i = \mathrm{K}[\mathrm{R}|\mathbf{t}]\mathbf{X}_i$~~
  - More general constraints: $\mathbf{x}_i = \mathrm{proj}(\mathbf{X}_i; \mathrm{R}, \mathbf{t})$ where $\mathbf{x}_i = [x_i, y_i]^\mathsf{T}$
    - Note) The projection proj generates 2D points on the image plane (not in homogeneous coordinates) considering nonlinear lens distortion.
  - Solutions ($n \geq 3$) → 3-point algorithm
    - OpenCV: `cv.solvePnP()` and `cv.solvePnPRansac()`
    - **Iterative 3-point algorithm** using local optimization
      - Cost function

$$\hat{\mathrm{R}}, \hat{\mathbf{t}} = \underset{\mathrm{R}, \mathbf{t}}{\operatorname{argmin}} \sum_{i=1}^{n} \|\mathbf{x}_i - \mathrm{proj}(\mathbf{X}_i; \mathrm{R}, \mathbf{t})\|_2^2$$

      - Optimizer: Gauss-Newton method



Solving Nonlinear Equations in 3D Vision

- **Absolute camera pose estimation** (perspective-n-point; PnP) [pose_estimation_implement.py]

```python
import numpy as np
from scipy.optimize import least_squares
from scipy.spatial.transform import Rotation
import cv2 as cv

def project_no_distort(X, rvec, t, K):
    R = Rotation.from_rotvec(rvec.flatten()).as_matrix()
    XT = X @ R.T + t                        # Transpose of 'X = R @ X + t'
    xT = XT @ K.T                           # Transpose of 'x = KX'
    xT = xT / xT[:,-1].reshape((-1, 1))  # Normalize
    return xT[:,0:2]

def reproject_error_pnp(unknown, X, x, K):
    rvec, tvec = unknown[:3], unknown[3:]
    xp = project_no_distort(X, rvec, tvec, K)
    err = x - xp
    return err.ravel()

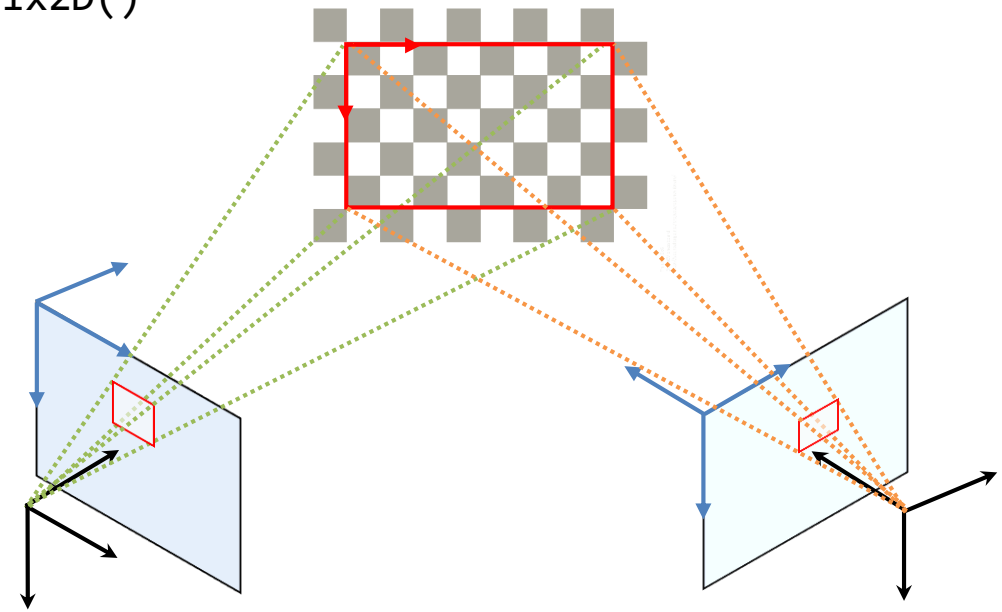
def solvePnP(obj_pts, img_pts, K):
    unknown_init = np.array([0, 0, 0, 0, 0, 1.]) # Sequence: rvec(3), tvec(3)
    result = least_squares(reproject_error_pnp, unknown_init, args=(obj_pts, img_pts, K))
    return result['success'], result['x'][:3], result['x'][3:]
```

Solving Nonlinear Equations in 3D Vision

- **Camera calibration**
  - Unknown: Intrinsic + m x extrinsic parameters (3* + m x 6 DOF)
  - Given: 3D points $\mathbf{X}_1$, $\mathbf{X}_2$, …, $\mathbf{X}_n$ and their projected points, $\mathbf{x}_i^j$, on the j-th image
    - Note) m: the number of images, n: the number of 3D points
  - ~~Constraints: m x n x projection $\mathbf{x}_i^j = \mathrm{K}\left[\mathrm{R}_j | \mathbf{t}_j\right]\mathbf{X}_i$~~
  - More general constraints: $\mathbf{x}_i^j = \mathrm{proj}\left(\mathbf{X}_i; \mathrm{K}, \mathrm{R}_j, \mathbf{t}_j\right)$ where $\mathbf{x}_i = [x_i, y_i]^\mathrm{T}$
  - Solutions [Tools]
    - OpenCV: `cv.calibrateCamera()` and `cv.initCameraMatrix2D()`
    - **Camera calibration** using local optimization
      - Cost function

$$\hat{\mathrm{K}}, \hat{\mathrm{R}}_{\dots}, \hat{\mathbf{t}}_{\dots} = \underset{\mathrm{K}, \mathrm{R}_{\dots}, \mathbf{t}_{\dots}}{\operatorname{argmin}} \sum_{j=1}^{m} \sum_{i=1}^{n} \left\| \mathbf{x}_i^j - \mathrm{proj}\left(\mathbf{X}_i; \mathrm{K}, \mathrm{R}_j, \mathbf{t}_j\right) \right\|_2^2$$

      - Optimizer: Gauss-Newton method



Solving Nonlinear Equations in 3D Vision

- **Camera calibration** [camera_calibration_implement.py]

```
import numpy as np
...

def fcxcy_to_K(f, cx, cy):
    return np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])

def reproject_error_calib(unknown, Xs, xs):
    K = fcxcy_to_K(*unknown[0:3])
    err = []
    for j in range(len(xs)):
        offset = 3 + 6 * j
        rvec, tvec = unknown[offset:offset+3], unknown[offset+3:offset+6]
        xp = project_no_distort(Xs[j], rvec, tvec, K)
        err.append(xs[j] - xp)
    return np.vstack(err).ravel()

def calibrateCamera(obj_pts, img_pts, img_size):
    img_n = len(img_pts)
    unknown_init = np.array([img_size[0], img_size[0]/2, img_size[1]/2] \
                 + img_n * [0, 0, 0, 0, 0, 1.]) # Sequence: f, cx, cy, img_n * (rvec, tvec)
    result = least_squares(reproject_error_calib, unknown_init, args=(obj_pts, img_pts))
    K = fcxcy_to_K(*result['x'][0:3])
    rvecs = [result['x'][(6*i+3):(6*i+6)] for i in range(img_n)]
    tvecs = [result['x'][(6*i+6):(6*i+9)] for i in range(img_n)]
    return result['cost'], K, np.zeros(5), rvecs, tvecs
```

Solving Nonlinear Equations in 3D Vision

- A camera lens generates geometric distortion, which can be approximated as a nonlinear function f_d.
- Geometric distortion models f_d are mostly defined on the normalized image plane.
- Camera projection with geometric distortion: $\mathbf{x} = \mathrm{proj}(\mathbf{X}; \mathrm{K}, \mathrm{R}, \mathbf{t}, d)$ where d is a set of distortion coefficients.

Note) $\mathbf{x} = \mathrm{K}(\mathrm{R}\mathbf{X} + \mathbf{t})$ without distortion and normalization

$$\mathbf{X} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \longrightarrow \mathbf{X}' = \mathrm{R}\mathbf{X} + \mathbf{t} \longrightarrow \hat{\mathbf{x}} = \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} = \begin{bmatrix} X'/Z' \\ Y'/Z' \end{bmatrix} \longrightarrow \hat{\mathbf{x}}_d = f_d(\hat{\mathbf{x}}) \longrightarrow \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} f_x \hat{x}_d + c_x \\ f_y \hat{y}_d + c_y \end{bmatrix}$$

3D point (the world coordinate) | 3D point (the camera coordinate) | 2D point (the normalized image plane) | **geometric distortion** | pinhole camera projection

$$\mathbf{x} = \mathrm{proj}(\mathbf{X}; \mathrm{K}, \mathrm{R}, \mathbf{t}, d)$$



- e.g. **Polynomial distortion model** (a.k.a. Brown-Conrady model; 1919)

$$\begin{bmatrix} \hat{x}_d \\ \hat{y}_d \end{bmatrix} = (1 + k_1 r^2 + k_2 r^4 + \cdots) \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} + (1 + p_3 r^2 + p_4 r^4 + \cdots) \begin{bmatrix} 2p_1 \hat{x}\hat{y} + p_2(r^2 + 2\hat{x}^2) \\ 2p_2 \hat{x}\hat{y} + p_1(r^2 + 2y^2) \end{bmatrix} \quad \text{where} \quad r^2 = \hat{x}^2 + \hat{y}^2$$

- **Camera calibration with lens distortion** [camera_calibration_implement_k1.py]

```python
def project_distort_k1(X, rvec, t, K, k1):
    R = Rotation.from_rotvec(rvec.flatten()).as_matrix()
    XT = X @ R.T + t                        # Transpose of 'X = R @ X + t'
    nT = XT / XT[:,-1].reshape((-1, 1))  # Normalize
    r2 = nT[:,0]**2 + nT[:,1]**2
    nT[:,0] = nT[:,0] * (1 + k1 * r2)
    nT[:,1] = nT[:,1] * (1 + k1 * r2)
    xT = nT @ K.T                           # Transpose of 'x = Kn'
    return xT[:,0:2]

def reproject_error_calib(unknown, Xs, xs):
    K = fcxcy_to_K(*unknown[0:3])
    k1 = unknown[3]
    err = []
    for j in range(len(xs)):
        offset = 4 + 6 * j
        rvec, tvec = unknown[offset:offset+3], unknown[offset+3:offset+6]
        xp = project_distort_k1(Xs[j], rvec, tvec, K, k1)
        err.append(xs[j] - xp)
    return np.vstack(err).ravel()

def calibrateCamera(obj_pts, img_pts, img_size):
    img_n = len(img_pts)
    unknown_init = np.array([img_size[0], img_size[0]/2, img_size[1]/2, 0.] + img_n * [0, 0, 0, 0, 0, 1.])
                # Sequence: f, cx, cy, k1, img_n * (rvec, tvec)
    result = least_squares(reproject_error_calib, unknown_init, args=(obj_pts, img_pts))
    K = fcxcy_to_K(*result['x'][0:3])
    dist_coeff = np.array([result['x'][3], 0, 0, 0, 0], dtype=np.float32)
    rvecs = [result['x'][(6*i+4):(6*i+7)] for i in range(img_n)]
    tvecs = [result['x'][(6*i+7):(6*i+10)] for i in range(img_n)]
    return result['cost'], K, dist_coeff, rvecs, tvecs
```

Summary) Solving 3D Vision Problems

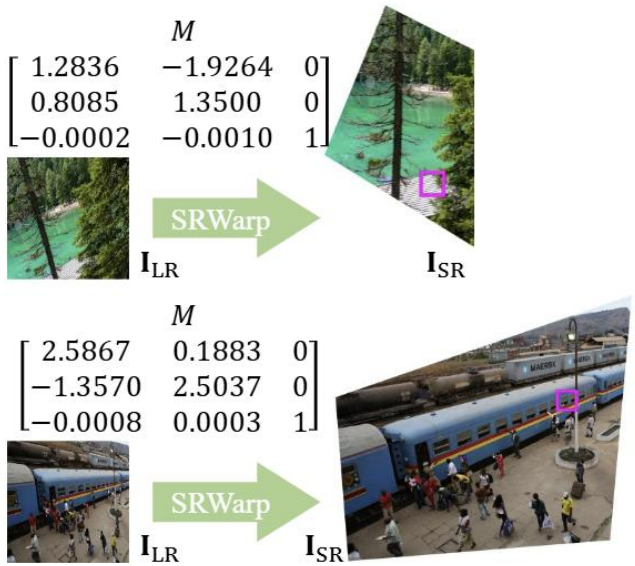
- **Linear equations**
  - Inhomogeneous equations $\mathbf{A}\mathbf{x} = \mathbf{b} \rightarrow \mathbf{x} = \mathbf{A}^{\dagger}\mathbf{b}$ where $\mathbf{A}^{\dagger}$ is a pseudo-inverse
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$
    - Example) Affine transformation estimation
  - Homogeneous equation $\mathbf{A}\mathbf{x} = \mathbf{0} \rightarrow \mathbf{x}$ is the last row of $\mathbf{V}^{\intercal}$ where $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^{\intercal}$ (from singular value decomposition)
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathbf{A}\mathbf{x}\|^2$ with $\|\mathbf{x}\|^2 = 1$ condition
    - Example) Planar homography estimation
    - Example) Fundamental matrix estimation
    - Example) Triangulation
- **Nonlinear equations**
  - Nonlinear optimization: Magic tools such as scipy.optimize and Ceres Solver
    - Formulation) $\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} f(\mathbf{x})$
    - Example) Absolute camera pose estimation (PnP)
    - Example) Camera calibration

Beyond Image Warping with Interpolation

- **Image warping with super-resolution**
  - e.g. SRWarp







(a) $\mathbf{I}_{LR}$, $\mathbf{I}_{SR}$, and M



(b) cv2 (c) RRDB + cv2 (d) **SRWarp** (e) GT

Image: Son et al., "SRWarp: Generalized Image Super-Resolution under Arbitrary Transformation", CVPR, 2021 arXiv Github

Beyond `scipy.optimize` and Ceres Solver for Nonlinear Least Squares

- **Nonlinear least squares on deep learning frameworks**
  - e.g. DeepLM (on PyTorch)
    - Automatic Jacobian calculation (← deep learning frameworks)
    - Stochastic decomposition (→ fast, memory-efficient)

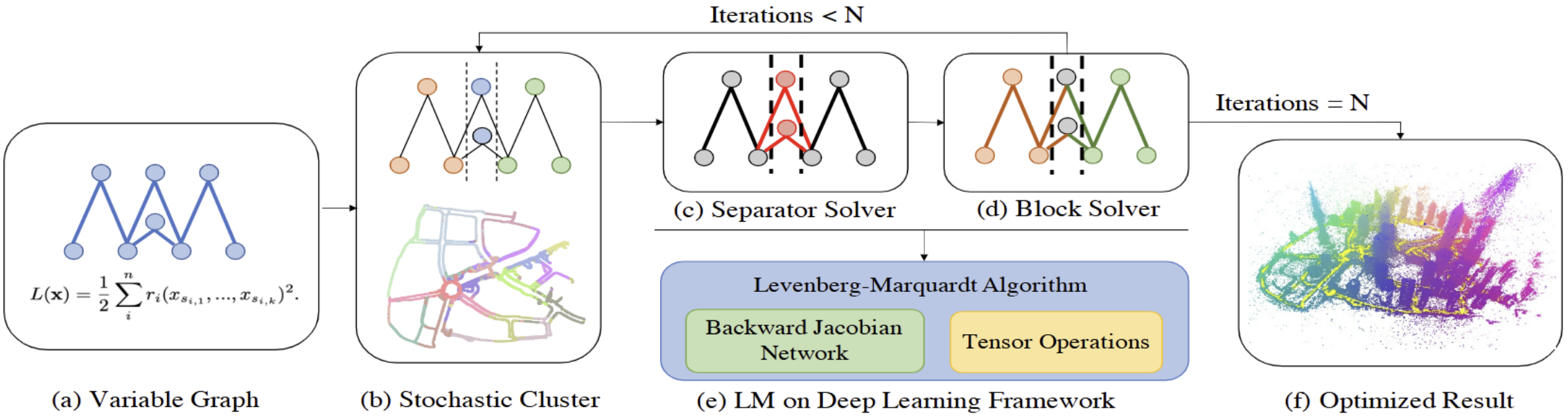


Image: Huang et al., "DeepLM: Large-scale Nonlinear Least Squares on Deep Learning Frameworks using Stochastic Domain Decomposition", CVPR, 2021 CVF Github